

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 07 -

CICLOS DE DEPLOY E AUTOMAÇÃO (CI/CD & MLOPS)

SUMÁRIO

O QUE VEM POR AÍ?3

HANDS ON4

SAIBA MAIS5

MERCADO, CASES E TENDÊNCIAS12

O QUE VOCÊ VIU NESTA AULA?13

REFERÊNCIAS.....14



O QUE VEM POR AÍ?

Chegamos ao momento de conectar engenharia de software, dados e operação em uma esteira única de entrega.

Nesta aula, você vai entender como construir ciclos de deploy automatizados para modelos de ML com Docker, Kubernetes e práticas de CI/CD orientadas por governança.

A proposta é mostrar como transformar o deploy em um processo confiável e auditável, reduzindo o risco de regressões e aumentando a velocidade de entrega sem perder controle de qualidade.

HANDS ON

Nessa aula, veremos uma esteira prática de entrega: build da imagem, validações automatizadas, publicação em registry e deploy em Kubernetes com estratégia controlada de rollout. Você verá como cada etapa reduz uma classe específica de risco operacional.

Durante a prática, a ênfase será em gates de qualidade e critérios de promoção entre ambientes, incluindo cenários de rollback quando os indicadores pós-deploy não atendem ao esperado. A ideia é simular o comportamento real de uma equipe de MLOps em produção.

Ao concluir, você terá uma visão clara de como organizar pipeline de ponta a ponta com rastreabilidade entre código, imagem e versão de modelo, base indispensável para escalar operações de ML.

SAIBA MAIS

CI/CD em ML: por que é mais complexo do que em software tradicional

Em software tradicional, a esteira de CI/CD já precisa garantir qualidade, segurança e velocidade. Em Machine Learning, além desses pontos, existe um fator adicional: o comportamento do sistema depende também do modelo e do contexto dos dados. Isso significa que uma pipeline madura precisa controlar tanto a dimensão de engenharia de software quanto a dimensão estatística do produto.

Em termos didáticos, pense em três perguntas que a pipeline precisa responder antes de promover uma versão:

- O código está tecnicamente correto e seguro?
- O artefato implantado é rastreável e reproduzível?
- O comportamento do modelo continua adequado ao objetivo de negócio?

Se uma dessas respostas for “não”, acelerar o deploy passa a ser risco, não vantagem.

Estrutura de pipeline madura para serviços de inferência

Uma esteira robusta normalmente tem as etapas build, testes, segurança, publicação de artefato imutável, deploy progressivo e validação pós-release. O ganho não está apenas em automatizar, mas em colocar critérios objetivos entre uma etapa e outra.

```
name: ml-ci-cd
on:
  push:
    branches: [ "main" ]
jobs:
  build-test-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t recomendacao-api:${{
github.sha }} .
```

```
      - run: pytest -q
      - run: trivy image --exit-code 1 recomendacao-api:${{
github.sha }}
    deploy:
      needs: build-test-scan
      runs-on: ubuntu-latest
      steps:
        - run: kubectl set image deployment/recomendacao-api
api=recomendacao-api:${{ github.sha }}
        - run: kubectl rollout status
deployment/recomendacao-api
```

Código-fonte 1 – Exemplo de pipeline simplificado (CI/CD) em manifesto de workflow (YAML)
Fonte: Elaborado pelo autor (2026)

Esse exemplo é simples, mas já explicita uma lógica essencial: não existe deploy sem validação técnica e de segurança.

A fase de CI precisa encontrar problemas cedo, quando o custo de correção é menor. Para APIs de inferência, recomenda-se incluir:

- Testes unitários e de integração.
- Validação de contrato da API (schema de entrada/saída).
- Verificação básica de desempenho.
- Análise de vulnerabilidades de imagem.

O objetivo é evitar que “erros baratos de corrigir” avancem para estágios caros, como homologação ou produção.

Entrega contínua com artefato imutável

A boa prática é promover o mesmo artefato entre ambientes. Em vez de reconstruir imagem em cada etapa, publica-se uma imagem única e imutável, alterando apenas configuração externa. Isso reduz divergência entre ambientes e facilita auditoria.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recomendacao-api
  labels:
```

```

  modelVersion: v3.4.1
  releaseId: "2026-03-08.1"
spec:
  replicas: 4
  selector:
    matchLabels:
      app: recomendacao-api
  template:
    metadata:
      labels:
        app: recomendacao-api
    spec:
      containers:
        - name: api
          image: registry.exemplo.com/recomendacao-api:sha-9f12ab3
```

Código-fonte 2 – Exemplo de Deployment com imagem imutável e metadados de release (YAML)
Fonte: Elaborado pelo autor (2026)

Esse padrão melhora a rastreabilidade: fica claro qual versão está em produção e como voltar rapidamente em caso de problema.

Toda mudança em produção envolve risco. A estratégia de rollout deve refletir esse risco.

ESTRATÉGIA	BENEFÍCIO	LIMITAÇÃO	CENÁRIO INDICADO
Rolling update	Simplicidade	Exposição gradual	Mudanças de baixo risco
Blue/Green	Reversão rápida	Custo temporário	Mudanças críticas
Canary	Validação progressiva	Exige métrica madura	Novas versões

Tabela 1 – Estratégias de rollout para diferentes níveis de risco
Fonte: Elaborado pelo autor (2026)

Para ML, o canary costuma ser valioso porque permite observar a qualidade de predição em uma pequena parcela de tráfego antes da promoção total.

Critérios de promoção e rollback orientados por dados

Promover versão sem critérios explícitos é decisão frágil. Critérios típicos incluem:

- Taxa de erro da API;
- Latência;
- Estabilidade de infraestrutura;
- Sinal de qualidade de inferência (quando disponível).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-v1
spec:
  replicas: 4
  selector:
    matchLabels:
      app: frontend
      version: "1"
  template:
    metadata:
      labels:
        app: frontend
        version: "1"
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-canary
spec:
  replicas: 1 # Apenas 1/5 do tráfego (20%) irá para cá
  selector:
    matchLabels:
      app: frontend
      version: "2"
  template:
    metadata:
      labels:
        app: frontend
        version: "2"
```



```
spec:
  containers:
    - name: nginx
      image: nginx:1.23 # Nova versão

apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  selector:
    app: frontend # Seleciona ambos (v1 e v2)
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

Código-fonte 3 – Exemplo de rollout canary em Kubernetes (duas versões coexistindo) (YAML)
Fonte: Elaborado pelo autor (2026)

Em MLOps, rastrear apenas commit não basta. Você precisa ligar:

- Código.
- Imagem docker.
- Versão de modelo.
- Configuração ativa.
- Ambiente de execução.

Sem essa cadeia, o post-mortem vira reconstrução manual de histórico e o tempo de resposta a incidentes aumenta.

Já a segurança não deve aparecer apenas no final da pipeline. Ela precisa estar distribuída no fluxo:

- Validação de dependências.
- Scanner de imagem.
- Assinatura/controle de artefatos.
- Revisão de permissões e segredos.

Em contextos regulados, esse nível de controle não é diferencial; é requisito mínimo para operar.

Pós-deploy e ciclo de aprendizado contínuo

A esteira não termina em “deploy com sucesso”. O valor real vem do acompanhamento pós-release para verificar se a hipótese da mudança se confirmou em produção.

Sem esse ciclo, a equipe automatiza entrega, mas não automatiza aprendizado. E, sem aprendizado, a qualidade tende a oscilar ao longo do tempo.

MATURIDADE	CARACTERÍSTICA	RISCO	PRÓXIMO PASSO
Básica	Build e deploy automáticos	Regressão sem controle	Incluir testes e scanner
Intermediária	Gates técnicos definidos	Rollback lento	Adotar rollout progressivo
Avançada	Promoção por métricas e rastreabilidade	Complexidade de operação	Automatizar aprendizado pós-release

Tabela 2 – Maturidade de pipeline em MLOps
Fonte: Elaborado pelo autor (2026)

Erros comuns de adoção e mitigação

- Erro comum 1: pipeline sem validação de qualidade mínima.
Correção: definir gates técnicos e operacionais por etapa.
- Erro comum 2: deploy direto sem estratégia progressiva.
Correção: usar canary/blue-green conforme risco.
- Erro comum 3: ausência de rastreabilidade entre modelo e imagem.
Correção: padronizar metadados obrigatórios de release.

CI/CD em MLOps é a disciplina que conecta velocidade, confiabilidade e governança. Quando build, validação, segurança, rollout e observabilidade operam juntos, o time entrega com mais segurança e reduz o risco de regressão em produção.

EMEND

MERCADO, CASES E TENDÊNCIAS

A adoção de CI/CD para Machine Learning evoluiu rapidamente nos últimos anos, principalmente em organizações que precisam publicar melhorias frequentes sem renunciar a rastreabilidade e conformidade. O mercado está migrando de pipelines pontuais, orientadas por scripts isolados, para esteiras padronizadas com gates de qualidade, segurança e governança de release.

Um caso real relevante é o trabalho conjunto da Volkswagen e da AWS, que descreve a construção de uma abordagem end-to-end de MLOps para escalar casos de uso industriais com mais consistência operacional. Veja mais [aqui](#).

O aprendizado principal para esta aula é que velocidade de deploy só se sustenta quando existe estrutura de validação e observabilidade ao longo de toda a cadeia.

As tendências mais fortes para os próximos ciclos são deploy progressivo por padrão (canary/blue-green), políticas automatizadas de promoção com base em métricas e integração mais estreita entre experimentação, registro de modelo e operação em produção. Em síntese, CI/CD em ML está deixando de ser diferencial e se tornando requisito de competitividade.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, consolidamos a visão de CI/CD aplicada ao contexto de MLOps, entendendo que automação eficiente depende de critérios técnicos e de governança, não apenas de execução automática de comandos.

Você viu como estruturar pipelines com gates de qualidade e segurança, trabalhar com artefatos imutáveis, aplicar estratégias de rollout progressivo e usar métricas para decidir promoção ou rollback com menos subjetividade.

Além disso, conectou rastreabilidade, observabilidade pós-deploy e aprendizado contínuo como parte do mesmo ciclo de entrega. Em termos práticos, isso significa sair de um deploy “rápido, porém arriscado” para um deploy “rápido, controlado e auditável”, capaz de sustentar evolução de modelos em produção com confiança.

REFERÊNCIAS

HUMBLE, J.; FARLEY, D. **Continuous Delivery**. Boston: Addison-Wesley, 2010.

KUENKEL, T. **Practical MLOps**. Birmingham: Packt, 2023.

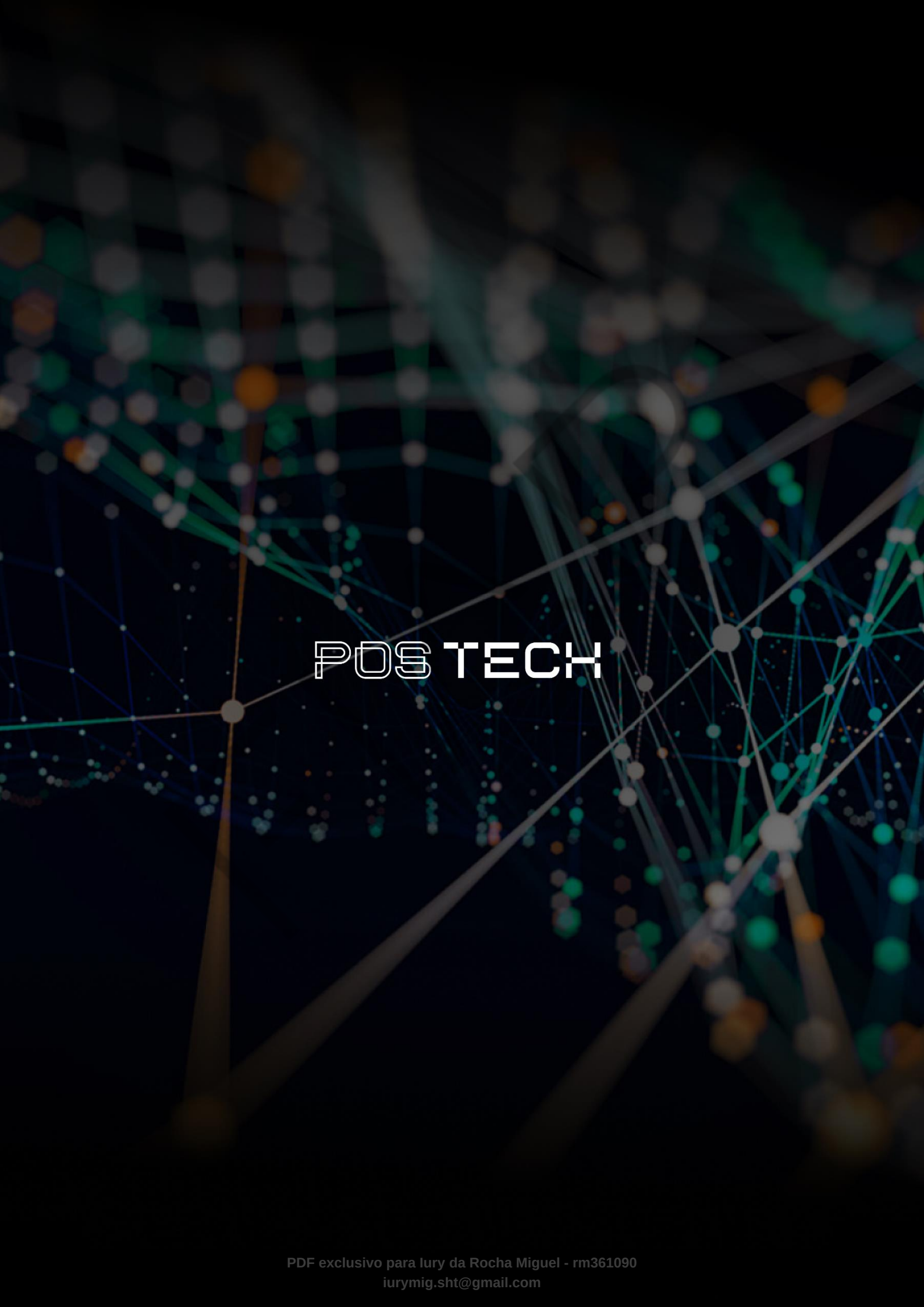
SATO, D. **DevOps na prática**. São Paulo: Casa do Código, 2014.

EMEND

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMSE



POSTECH